

Task 2

Hands-On Machine Learning: Chapter 2

Introduction:

Chapter 2 focuses on step by step process of building complete ML project. It covers theoretical and implementation of each process building ML project. This report gives a brief overview of chapter 2 and why, how, and what impact does the described steps creates on ML operations.

Workflow of End to End ML Project:

Step 1: Look at the bigger picture:

This step makes us to get gist of the ML problem we are going to solve. Finding and framing the problem tells us about the objective of building a specific model and how it is going to used in real life. Which algorithm will we select, which evaluation metrics can evaluate our model more effectively and efficiently.

In book, it uses California census data to build a model of housing prices in the state. This data includes metrics such as the population, median income, and median housing price for each block group in California.

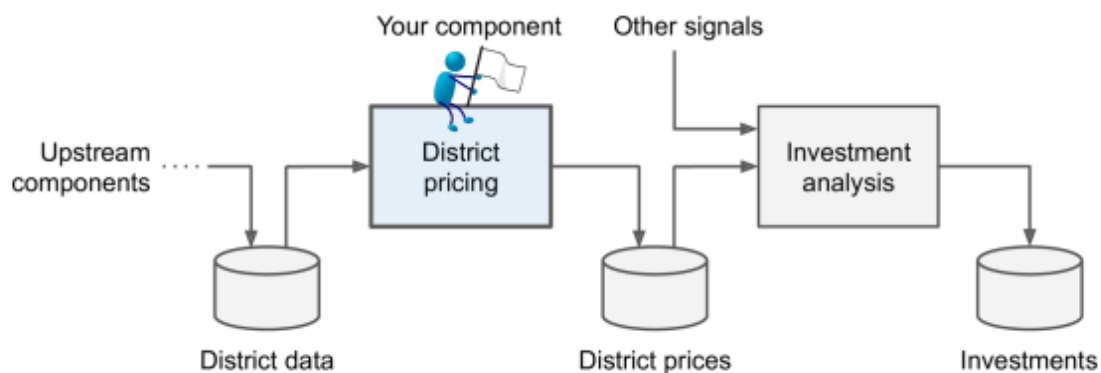


Figure 2-2. A Machine Learning pipeline for real estate investments

$$\text{RMSE}(X, h) = \sqrt{\frac{1}{m} \sum_{i=1}^m \left(h(x^{(i)}) - y^{(i)} \right)^2}$$

Methods and techniques:

- Identify business need.
- Identify which ML technique can solve this problem.
- Identify dependent variables (label) and independent variables (features).
- Identify which performance metrics can do best evaluation.

Importance of this step:

- Ensure that the problem going to be solved is aligned with our objectives.
- Makes sure that right ML technique is chosen, incorrect framing can leads to inaccurate and poor results.
- Defines how can we efficiently measure our outcomes. i.e choosing best evaluation metrics.

Step 2: Get the data:

In this step, we will get data from different sources. It can be in different format. Eg: CSV, PDF, Image, etc. In book data is in CSV format. And a function is defined to download the data for syncing with regular changes in data.

After downloading data from source it will be better to **Take fa Quick Look at the Data Structure**. This gives the overall insight of data, what are the values of features, and data is organized.

Housing is where the dataset is loaded in book.

We can use following techniques to lookup the data:

- i. ***Housing.head()*** : provide quick lookup to first 10 row of data.
- ii. ***Housing.info()*** : get a quick description of the data, in particular the total number of rows, each attribute's type, and the number of nonnull values.
- iii. ***Housing.describe()*** : shows common statistical parameters like:- count, mean, standard deviation, min, max.

- iv. Visualize statistical parameters with histogram or any other graph quickly.
- v. Identify datatypes (numerical, categorical).

While **Creating a Test Set**, we shouldn't take data from a certain part of dataset, this may lead to error and will not perform as we expected. Also, when we take test set from another part of same dataset then it may give different result than previous although both tests are from same dataset. Which is called *data snooping bias*.

So, we will take data randomly to cover most of the variances in dataset.

```
train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42).
```

Scikit-Learn provides a few functions to split datasets into multiple subsets in various ways. The simplest function is `train_test_split()`, which does pretty much the same thing as the function `split_train_test()`, with a couple of additional features. First, there is a `random_state` parameter that allows you to set the random generator seed. Second, you can pass it multiple datasets with an identical number of rows, and it will split them on the same indices (this is very useful, for example, if you have a separate DataFrame for labels).

Step 3: Discover and Visualize the Data to Gain Insights:

In this step, we will go deeper into our dataset. Data exploration is done only on training not in test set. If training set is large, then we can take a sample from training set and explore it. Data exploration is mostly done by plotting a feature against other features to find how much are they correlated with.

We can use following plots to visualize data to gain insights: -

- Scatter plot
- Correlation Matrix
- Scatter matrix , etc.

When we are done with exploring dataset we will **Experiment with different Attribute Combinations**. Since, We are already known that which features are highly correlated with each other. We combine features to find or make new features what can be highly correlated with dependent features or target variables. This process of identifying and making new features is **Feature Engineering**.

Step 4: Prepare the Data for Machine Learning Algorithms:

This is the most crucial part of ML process.

Reason of doing this are:

- This will allow you to reproduce these transformations easily on any dataset (e.g., the next time you get a fresh dataset).
- You will gradually build a library of transformation functions that you can reuse in future projects.
- You can use these functions in your live system to transform the new data before feeding it to your algorithms.
- This will make it possible for you to easily try various transformations and see which combination of transformations works best.

Data Cleaning is done to handle missing data, handle data with irrelevant information and place relevant data. Because misleading data can leads us to poor results.

Since, ML model are not good with **handling text and categorical features** we have to convert them into numerical form without losing information.

OrdinalEncoder(),OneHotEncoder() are used to do that.

We create a custom transformers to works seamlessly with pipelines. We create a class with three methods: fit() (returning self), transform(), and fit_transform(). You can get the last one for free by simply adding TransformerMixin as a base class. If you add BaseEstimator as a base class (and avoid *args and **kwargs in your constructor), you will also get two extra methods (get_params() and set_params()) that will be useful for automatic hyperparameter tuning.

Feature scalling is most important transformation. In dataset we can have a wide range of columns with different range of values. That can skew our patter. So we have to scale it for better results.

Min max scaling and standardization are used to perform feature scaling.

Minmax scaling: values are shifted and rescaled so that they end up ranging from 0 to 1. We do this by subtracting the min value and dividing by the max minus the min

Standardization: first it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the standard deviation so that the resulting distribution has unit variance. Unlike min-max scaling, standardization does not bound values to a specific range, which may be a problem for some algorithms (e.g., neural networks often expect an input value ranging from 0 to 1)

Step 5: Select and train a model:

After completing the preparation of dataset we select a relevant model and train on our cleaned data.

In book linear regression is used with RMSE evaluation.

We can do Cross- Validation for better result. which means

A great alternative is to use Scikit-Learn's K-fold cross-validation feature. The following code randomly splits the training set into 10 distinct subsets called folds, then it trains and evaluates the Decision Tree model 10 times, picking a different fold for evaluation every time and training on the other 9 folds. The result is an array containing the 10 evaluation scores.

Step 6: Fine tune your model:

To make our model work on our data more better we should finetune our model with our data.

Grid search can be one option to be fiddle with hyperparameters manually until you get combination of hyperparameter values. This would be very tedious work, and you may not have time to explore many combinations..

Randomized Search:

The grid search approach is fine when you are exploring relatively few combinations, like in the previous example, but when the hyperparameter search space is large, it is often preferable to use `RandomizedSearchCV` instead. This class can be used in much the same way as the `GridSearchCV` class, but instead of trying out all possible combinations, it evaluates a given number of random combinations by selecting a random value for each hyperparameter at every iteration.

Then we evaluate our model with test set and lookup to results, if results are not satisfactory then we do steps from steps 5.

Step 7: Launch, Monitor, and Maintain Your System

Atlast, when our model is ready to use, we launch our model. We can deploy our model to our production environment.

One way to do this is to save the trained Scikit-Learn model (e.g., using `joblib`), including the full preprocessing and prediction pipeline, then load this trained model within your production environment and use it to make predictions by calling its `predict()` method.

Another popular strategy is to deploy your model on the cloud, for example on Google Cloud AI Platform (formerly known as Google Cloud ML Engine).

But deployment is not the end of the story. You also need to write monitoring code to check your system's live performance at regular intervals and trigger alerts when it drops. This could be a steep drop, likely due to a broken component in your infrastructure, but be aware that it could also be a gentle decay that could easily go unnoticed for a long time. This is quite common because models tend to “rot” over time: indeed, the world changes, so if the model was trained with last year's data, it may not be adapted to today's data.